

Clausario

Scoring Guide

Compliance, tested before it's questioned.

Review agent score and evaluation · November 2026

> **Educational use only.** Proof of concept, not production-ready, not legal advice and no compliance certification for the GDPR, the EU AI Act or any other law. Use synthetic data only. Provided "as is", without warranty of any kind.

© 2026 Sudarshan Ramanujam. All rights reserved. Regulation text © European Union.

> **Educational use only.** Proof of concept, not production-ready, not legal advice and no compliance certification. See the full notice in README.md.

How the Review agent score and the scenario evaluation are calculated.

Contents

1. How scoring works
2. Review agent score
3. Evaluation
4. Settings and where to look

1. How scoring works

Both scores follow one rule: **the decision engine (OpenAI Decisions API or Jev) only answers questions with probabilities, and Clausario's code turns those answers into the score.** No LLM writes a score or a verdict.

The engine answers three kinds of question:

Type	What the engine returns	Example
Choice	the option it picks, plus its confidence (0–1)	"GDPR, EU AI Act, both or none?" → GDPR, 0.90
Score	a probability for each level: Low, Medium, High	Low 0.1 · Medium 0.3 · High 0.6
Predicate	the probability that a statement is true (0–1)	"Are these articles relevant?" → 0.80

The engine may also **refuse** a question. A refused question never counts in a run's favour: it adds 0 to the Review score, and it turns an evaluated scenario into *warn*.

2. Review agent score

The Review agent runs after article selection and gives a score from 0 to 100. The engine reads the story and the selected articles, then answers four questions.

Part	Question asked	How the code turns the answer into 0–1	Weight
Agreement	`reg_check` (choice): which regulation applies?	If its pick matches the routed regulation: its confidence. If not: 1 − confidence	**35**
Specificity	`story_specific` (score): how specific is the story about the data, the people and the system behaviour?	Low = 0, Medium = 1, High = 2, weighted by probability, then ÷ 2	**15**
Relevance	`articles_relevant` (predicate): are the selected articles obligations or rights to test against?	The probability as it is	**30**
Completeness	`article_missing` (predicate): is an obviously applicable article missing?	1 − probability	**20**

```
Score = round(35 × Agreement + 15 × Specificity + 30 × Relevance + 20 × Completeness)
```

Worked example

A story routed to GDPR:

Part	Engine answer	Value	× Weight
Agreement	GDPR, confidence 0.90 (matches the route)	0.90	31.5
Specificity	Low 0.1 · Medium 0.3 · High 0.6 → (0 × 0.1 + 1 × 0.3 + 2 × 0.6) ÷ 2	0.75	11.25
Relevance	0.80	0.80	24
Completeness	missing = 0.30 → 1 − 0.30	0.70	14
Score			**81**

If the engine had picked "both" with confidence 0.80 instead, Agreement would be 1 − 0.80 = 0.20, and the score would drop by about 25 points.

What happens next

Score	Result
≥ `REVIEW_THRESHOLD` (70 by default)	The run continues to scenario generation on its own
< `REVIEW_THRESHOLD`	The run pauses in the **Review queue** for a reviewer

The reviewer can:

- **Approve:** generate scenarios with the current articles.
- **Override:** change the regulations or articles, then generate.
- **Add context:** add the missing facts. The same four questions are asked again and a new score is calculated, up to `HITL_MAX_ROUNDS` times (3 by default).
- **Reject:** end the run as *Needs rework*.

Vague stories score low because Specificity is low and Agreement is uncertain, so they end up in the queue.

3. Evaluation

Evaluation runs after generation, when it is turned on. In one decision call, the engine reads the story, the regulation text the scenarios were generated from (the context pack) and the scenarios as JSON. It returns a verdict per scenario and a coverage figure for the run.

Questions per scenario

Question	Type	Asks
Traceability	score	Does it directly test the obligation in the clauses it cites?
Testability	score	Does it have concrete steps and an observable expected result?
Clarity	score	Is it unambiguous, and about one behaviour (not several bundled together)?
Priority fit	choice	Is its High / Medium / Low priority too low, correct or too high?
Duplicate	predicate	Does it test the same behaviour as another scenario?

Question per selected article

Question	Type	Asks
Coverage	predicate	Does at least one scenario test an obligation or right of this article?

From level probabilities to a number

Each of the three score questions becomes a number from 0 to 2, weighted by probability and rounded to two decimals:

`level score = (0 × P(Low) + 1 × P(Medium) + 2 × P(High)) ÷ total probability`

For example, Low 0.05 · Medium 0.25 · High 0.70 gives **1.65**.

Verdict rules

Checked in this order, shown with the default `.env` values:

Verdict	When
fail	any of the three scores is below `EVAL_SCORE_FAIL` (0.7)
warn	any score is below `EVAL_SCORE_PASS` (1.3), **or** priority fit is not "correct", **or** the duplicate probability is ≥ `EVAL_DUP_THRESHOLD` (0.7), **or** the engine refused one of the scenario's questions
pass	none of the above

Worked example

Item	Engine answer	Result
Traceability	Low 0.05 · Medium 0.25 · High 0.70	1.65
Testability	Low 0.10 · Medium 0.60 · High 0.30	1.20
Clarity	Low 0.00 · Medium 0.50 · High 0.50	1.50
Priority fit	"correct"	OK
Duplicate	0.10	OK
Verdict	Testability 1.20 is below 1.3	**warn**

Nothing is below the fail line (0.7), so the scenario is not *fail*, but one weak score is enough for *warn*.

Coverage

An article is **covered** when its coverage probability is at least `ARTICLE_THRESHOLD` (0.5).

```
Coverage % = covered articles ÷ selected articles × 100, rounded
```

The Evaluation tab shows the pass, warn and fail counts, and coverage, for example "Coverage · 8/8 articles".

4. Settings and where to look

Settings (`backend/.env`)

Setting	Default	Used by
`REVIEW_THRESHOLD`	70	Review: the score needed to continue without a reviewer
`HITL_MAX_ROUNDS`	3	Review: how many times a reviewer can add context
`REVIEW_ENGINE`, `EVAL_ENGINE`	`DECISION_ENGINE`	Which engine (Decisions or Jev) answers each step
`EVAL_ENABLED`	true	Turns evaluation on or off
`EVAL_SCORE_PASS`	1.3	Evaluation: lowest score for *pass*
`EVAL_SCORE_FAIL`	0.7	Evaluation: below this is *fail*
`EVAL_DUP_THRESHOLD`	0.7	Evaluation: duplicate probability that gives *warn*
`ARTICLE_THRESHOLD`	0.5	Evaluation: coverage probability that counts as covered (also used in article selection)

The Review weights (35 / 15 / 30 / 20) are fixed in code.

Code

What	File
Review questions, weights and score	`backend/src/graph/reviewScore.ts`
Pause, resume and reviewer actions	`backend/src/graph/graph.ts`
Evaluation questions, level score, verdict and coverage	`backend/src/decisions/evaluate.ts`
Calling the engine, retries and fallbacks	`backend/src/decisions/client.ts`, `backend/src/decisions/engines/runnable.ts`

In the app

Screen	Shows
Run detail → **Review** tab	The four parts, their values, the score and the review log
Run detail → **Evaluation** tab	Pass, warn and fail counts, and coverage
Run detail → **Scenarios** tab	Each scenario's verdict badge
Run detail → **Cost** tab	Tokens and cost for the review and evaluation steps

Things to know

- **The thresholds are not calibrated yet.** With *pass* at 1.3, many real scenarios get *warn*, because testability often comes out around 0.8–1.1. Tune the values in `.env` after a few real runs.
- **Engines differ.** Jev's probabilities may sit differently from the Decisions API, so check the thresholds again if you switch engines.
- **Scores support people, they don't replace them.** A *pass* or a high Review score is a suggestion; a qualified person should still check the scenarios before relying on them.